

DESAFIO PRÁTICO - Aula 13 - Automação de Testes e Testes Unitários com PHPUnit

Etapa 1: O Código a ser Testado (Regras de Negócio)

Crie o arquivo `Loja.php` dentro da pasta `src\` do seu projeto e cole o código abaixo. Esta é a classe do nosso sistema de E-commerce que contém as regras que vocês deverão testar.

PHP

```
<?php
```

```
// src/Loja.php
```

```
class Loja {
```

```
    // Regra 1: Cálculo matemático com porcentagem
```

```
    public function calcularDesconto(float $valorOriginal, float $percentualDesconto) : float {  
        $desconto = $valorOriginal * ($percentualDesconto / 100);  
        return $valorOriginal - $desconto;  
    }
```

```
    // Regra 2: Retorno Booleano baseado em limite (Frete Grátis acima de R$ 200)
```

```
    public function temFreteGratis(float $valorDaCompra) : bool {  
        return $valorDaCompra >= 200.0;  
    }
```

```
    // Regra 3: Validação que lança uma Exceção em caso de erro
```

```
    public function solicitarEstoque(int $quantidadeSolicitada, int $estoqueDisponivel) : bool {  
        if ($quantidadeSolicitada <= 0) {  
            throw new InvalidArgumentException("A quantidade solicitada deve ser maior que zero.");  
        }  
        return $quantidadeSolicitada <= $estoqueDisponivel;  
    }
```

```
    // Regra 4: Adição de taxa fixa de manuseio
```

```
    public function aplicarTaxaDeManuseio(float $valor) : float {  
        return $valor + 15.50;  
    }
```

```
    // Regra 5: Condicionais (Categorização de cliente)
```

```
    public function classificarCliente(float $totalGasto) : string {  
        if ($totalGasto >= 1000.0) {  
            return 'VIP';  
        } elseif ($totalGasto >= 500.0) {  
            return 'Ouro';  
        }  
        return 'Prata';  
    }
```

}

Etapa 2: Exercícios Práticos

Instrução: Crie o arquivo *LojaTest.php* dentro da pasta *tests*. Dentro dessa classe de teste (que deve herdar de *TestCase*), escreva um método de teste para cada um dos exercícios abaixo utilizando obrigatoriamente o padrão **Triple A** (Arrange, Act, Assert).

Exercício 1: Validar o Cálculo de Desconto

Crie o método `testCalcularDescontoComSucesso()`.

Objetivo: Simular um produto que custa R\$ 100,00 com 20% de desconto. Verifique se o método `calcularDesconto` retorna exatamente R\$ 80,00. Utilize a asserção `$this->assertEquals()`.

Exercício 2: Validar o Frete Grátis

Crie o método `testDeveRetornarFreteGratisParaComprasAcimaDe200()`.

Objetivo: Simular uma compra no valor de R\$ 250,00. O método `temFreteGratis` deve retornar um valor booleano verdadeiro. Utilize a asserção `$this->assertTrue()`.

Exercício 3: Validar a Proteção contra Quantidade Inválida (Exceção)

Crie o método `testSolicitarEstoqueInvalidoLancaExcecao()`.

Objetivo: Tentar solicitar uma quantidade igual a zero (0) de um produto onde há 10 no estoque. O PHPUnit deve esperar que uma exceção do tipo `InvalidArgumentException` seja lançada pela classe *Loja*.

Dica: Lembre-se que no PHPUnit a asserção de exceção (`$this->expectException`) vem ANTES da ação (Act).

Exercício 4: Validar a Taxa de Manuseio

Crie o método `testAplicarTaxaDeManuseioAdicionaValorFixo()`.

Objetivo: Simular um produto de R\$ 50,00. Verifique se o método `aplicarTaxaDeManuseio` soma corretamente os R\$ 15,50 fixos, devendo retornar R\$ 65,50. Utilize a asserção `$this->assertEquals()`.

Exercício 5: Validar a Classificação de Cliente VIP

Crie o método `testClassificarClienteVipComSucesso()`.

Objetivo: Simular um cliente que gastou um total de R\$ 1.200,00 na loja. O método `classificarCliente` deve retornar a string 'VIP'. Utilize a asserção `$this->assertEquals()`.

Etapa 3: Documentação dos Casos de Teste

Objetivo: Consolidar a rastreabilidade entre o que o sistema deve fazer e o que foi codificado, desenvolvendo o raciocínio analítico de Qualidade de Software (QA).

Instruções:

Agora que você já desenvolveu e executou os testes automatizados utilizando o PHPUnit, é necessário documentar formalmente esses cenários. Crie um documento (pode ser uma planilha no Excel/Google Sheets ou uma tabela no Word/Google Docs) e elabore um **Caso de Teste** para cada um dos 5 exercícios resolvidos na Etapa 2.

A sua tabela deve conter obrigatoriamente as seguintes colunas:

- **ID do Caso:** Um identificador único (ex: CT-001, CT-002).
- **Título / Objetivo:** O que este teste está validando.
- **Pré-condições:** O que precisa estar pronto antes do teste rodar (ex: "Classe Loja instanciada").
- **Passos para Execução:** O passo a passo lógico para reproduzir a ação.
- **Dados de Entrada:** Os valores exatos que você utilizou no seu Arrange.
- **Resultado Esperado:** O que o sistema deve retornar caso funcione perfeitamente.
- **Resultado Obtido:** O que o sistema de fato retornou após você rodar o PHPUnit.
- **Estado:** Aprovado ou Reprovado.
- **Script Associado:** O nome do método no seu arquivo LojaTest.php (ex: testCalcularDescontoComSucesso).

Dica: Utilize este modelo como base para preencher os dados referentes a todos os 5 exercícios.

Etapa 4: Instruções de Entrega do Material

Objetivo: Organizar os artefatos desenvolvidos para avaliação técnica da estrutura do código e da documentação.

O que deve ser entregue:

Você deverá compilar o seu trabalho e enviar no AVA o seguinte pacote de arquivos (compactados em um arquivo .zip):

1. Código-Fonte dos Testes (LojaTest.php):

- O arquivo contendo a classe de teste com os 5 métodos construídos obrigatoriamente no padrão *Triple A*.

2. Evidência de Execução (Print de Tela):

- Uma captura de tela (screenshot) nítida do seu terminal (CMD, PowerShell ou VS Code) mostrando o comando de execução do PHPUnit e o feedback visual de sucesso (os pontos verdes e a mensagem *OK (5 tests, 5 assertions)*).

3. Documento de Casos de Teste:

- O arquivo (PDF ou Planilha) elaborado na Etapa 3, contendo a matriz de testes devidamente preenchida para os 5 cenários.

Critérios de Avaliação:

- A exatidão da sintaxe e o uso correto das asserções do PHPUnit (assertEquals, assertTrue, expectException).
- A correta separação estrutural do padrão *Triple A* dentro de cada teste.
- A coerência entre a documentação do Caso de Teste (Etapa 3) e o código automatizado (Etapa 2).